ARIA API / VAIS – Short Setup Guide up to the Final Test

1. Order and installation

Start with the Varian 0 € order for ARIA API / VAIS. After installation, the basic API components should be available on the ARIA / Shared Framework server.

2. Check the actual local endpoints

Do not rely blindly on the ports shown in the documentation. In practice, the URLs can be confusing or installation-specific.
In the working setup, the relevant pattern was:

```
Token service:
https://<sf-server>:44333/tokenservice/connect/token

FHIR base:
https://<sf-server>:55370/fhir/r4

API Tester / Conformance:
https://<sf-server>:44370/tester/conformance?serverId=<server-id>&apiKey=&pretty=false

Documentation / Tester entry:
https://<sf-server>:55370/document/home.html
```

The conformance statement shows which FHIR resources, interactions, search parameters and operations are available in the current installation.
The TokenUrl and FhirBase are used for scripted access, for example with PowerShell, Python or C#. Tokens generated by script can also be used in the web tester to experiment with API calls.

3. Obtain a Client ID and Secret

After installation, you need a Client ID and Secret.
The installation may include a standard client such as:

`VARIAN_API_VAIS_CLIENT`

This client can be useful for basic testing, but it may have only a minimal scope. In our case, the initial client was limited to:

`system/DocumentReference.cruds`

That was enough to verify that token generation and basic API access work, but not enough for practical administrative API work.

4. Create or extend a VAIS client

For real use, either extend the existing client or create a dedicated Headless Client.
This can be done in the VAIS Administration Portal. If local users do not have access to that portal, request it through a MyVarian ticket.
For the ticket, provide:

- **ClientName**
- **ARIA / DataAdmin user details:** DisplayName, LastName, FirstName
- **Secret:** at least 20 characters
- **Expiration date:** for example maximum allowed validity or a fixed period
- **Requested scope:** specific or full/extended scope

For administrative or API development work, request the required full or extended scope explicitly. Scopes are not automatically available just because they appear in the documentation.

5. Scope handling

For basic document-only testing, DocumentReference may be enough.
For useful administrative/API work, the client should have access to broader system/... and, if needed, user/... scopes. Otherwise, important steps such as patient lookup, organization

lookup, ValueSet expansion and general FHIR exploration are not possible.
The standard installation client may therefore be useful for a first check, but a dedicated client is usually required for real work.

6. Run the TokenTester script

Use the attached PowerShell TokenTester script (aria-fhir-quicktest.ps1) after receiving the new client credentials.
The script should verify:

- Token without explicit scope
- Each individual scope
- The complete scope string
- FHIR test requests using the generated token

In the final working setup, all tested system/... and user/... scopes were accepted with the new client.
In the script, replace sensitive or local values with your environment-specific values:

```
$ClientId     = "your-client-id-here"
$ClientSecret = "your-client-secret-here"
$TokenUrl     =
"https://<aria-server>:44333/tokenservice/connect/token"
$FhirBase     = "https://<aria-server>:55370/fhir/r4"
```

Note: Do not store real secrets in shared scripts or repositories.

7. Final functional test

Once token generation and scope tests are successful, perform a functional API test through either the API Tester web page or your own script (like the provided aria-fhir-quicktest.ps1).

Important note on FHIR IDs:
When calling specific resources, the FHIR ID is constructed using the ARIA serial number (not the standard ARIA ID). For example, a Patient FHIR ID must be formatted as

`Patient-<ARIA_Serial_Number>.`

Recommended final checks:

- Conformance / metadata
- Patient search or read
- Organization search
- ValueSet expansion
- DocumentReference search or write, depending on the intended use

The API Tester is useful for exploring possible requests interactively. Scripts are better for reproducible testing and later automation.

8. Start productive scripting

After the final tests are successful, continue with your own C#, Python or PowerShell scripts against the ARIA API.

At this point, the important pieces are in place:

- ARIA API / VAIS installed
- Correct local endpoints identified
- Dedicated Client ID available
- Secret available
- Required scopes assigned
- Client configured in ARIA / DataAdmin
- Token generation tested
- FHIR calls tested (e.g., using aria-fhir-quicktest.ps1)
- API Tester available for exploration